

Struct

Mettiamo in ipotesi di voler rappresentare i dati di una persona; questi includono, ad esempio, nome, cognome, anno di nascita o età, codice fiscale, e possono essercene molti altri. Tutti questi dati sono in quale modo legati tra di loro, in quanto appartengono ad una singola persona.

Le struct rappresentano un modo per raggruppare variabili diverse che hanno una qualche relazione. Ogni variabile della struct è detta membro della struct.

Sintassi per creare una struct:

```
struct nomeStruttura {  
    tipo variabile1;  
    tipo variabile2;  
    ...  
};
```

Per poter accedere ad una struttura è necessario creare una variabile che la rappresenti; per poterlo fare è sufficiente, nel main, usare ancora la parola chiave struct, seguita dal nome della struttura e, a seguire, il nome della variabile.

Esempio:

```
struct miaStruttura{  
    int mioNumero;  
    char miaLettera;  
}  
  
int main(){  
    struct miaStruttura s;  
  
    return 0;  
}
```

In questo modo abbiamo creato una struttura chiamata "miaStruttura" ed s è una variabile che rappresenta questa struttura.

Chiaramente ora sorge spontanea una domanda: come possiamo accedere ai singoli membri della struttura che abbiamo creato?

Per farlo utilizziamo la seguente sintassi:

nomeStruttura.membro

Esempio:

```
struct miaStruttura{
    int mioNumero;
    char miaLettera;
}

int main(){
    struct miaStruttura s;

    s.mioNumero = 5;
    s.miaLettera = 'a';

    return 0;
}
```

Così facendo abbiamo assegnato un valore ai singoli campi (alle singole variabili) della struttura rappresentata dalla variabile s.

Un altro modo per poter assegnare un valore alle variabili di una struttura è farlo durante la fase di dichiarazione della stessa. Per farlo è sufficiente inserire l'elenco dei valori, separati da una virgola.

N.B.: chiaramente è opportuno inserire i valori rispettando l'ordine dei tipi delle variabili che compaiono all'interno della struttura.

Esempio:

```
struct miaStruttura{
    int mioNumero;
    char miaLettera;
}

int main(){
    struct miaStruttura s = {5, 'a'};

    return 0;
}
```

Osserviamo come questa assegnazione sia più semplice da fare rispetto alla precedente.

Ovviamente possiamo modificare i valori iniziali, ma per farlo è necessario utilizzare la sintassi precedente, ovvero usando il punto.

Naturalmente è possibile avere degli array all'interno della nostra struttura e questi verranno gestiti esattamente come abbiamo già visto.

Una situazione particolare è rappresentata dalle stringhe: è vero che queste sono degli array, per la precisione array di caratteri; tuttavia, non possiamo utilizzare l'operatore punto in questo modo:

```
struct miaStruttura{
    int mioNumero;
    char miaLettera;
    char miaStringa[10];
}

int main(){
    struct miaStruttura s;

    s.miaStringa = "Ciao";

    return 0;
}
```

Se proviamo a fare una operazione del genere ci viene restituito un errore.

Per poter assegnare un valore ad una stringa è necessario utilizzare la funzione strcpy.

Esempio:

```
struct miaStruttura{
    int mioNumero;
    char miaLettera;
    char miaStringa[10];
}

int main(){
    struct miaStruttura s;

    strcpy(s.miaStringa, "Ciao");

    return 0;
}
```

Così facendo non abbiamo alcun tipo di errore.

Tuttavia, non è necessario utilizzare strcpy se assegniamo un valore in fase di dichiarazione.

Esempio:

```
struct miaStruttura{
```

```

    int mioNumero;
    char miaLettera;
    char miaStringa[10];
}

int main(){
    struct miaStruttura s = {5, 'a', "Ciao"};

    return 0;
}

```

Questo esempio è perfettamente valido.

Un'altra operazione possibile è assegnare il valore di una struttura ad un'altra.

Esempio:

```

struct miaStruttura{
    int mioNumero;
    char miaLettera;
    char miaStringa[10];
}

int main(){
    struct miaStruttura s1 = {5, 'a', "Ciao"}, s2;

    s2 = s1;

    return 0;
}

```

In questo esempio s2 sarà una struttura con gli stessi valori di s1.

Per evitare di dover ripetere tutte le volte la parola chiave **struct** quando si dichiarano le variabili, usiamo la seguente sintassi:

```

typedef struct{
    int mioNumero;
    char miaLettera;
    char miaStringa[10];
}miaStruttura;

```